```
    # This method is private
  end

  public
  # subsequent methods will be "public"
  def method4
    # this method is public
  end
end
```

Since the default access for methods is public, it's rare to use public explicitly to denote access control.

As a matter of style, the methods after the call to an access method like public are typically not indented—you aren't defining a block, just the access status of subsequent methods.

Alternatively, you can set access levels of named methods by listing them as arguments to the access control functions:

```
class MyClass
  def method1
  end

  def method2
  end
  # ... and so on

  public :method1, :method4
  protected :method2
  private :method3
end
```

This mechanism is somewhat rare in practice, but it does enable the third way to declare access in Ruby.

We've mentioned that most statements in Ruby return a value. In particular, defining a method with def returns a value—the name of the new method as a symbol. As a result, you can declare access directly preceding a method definition.

```
class MyClass
  def method1
    # This method is public
  end

  protected def method2
    # This method is protected
  end

  private def method3
    # This method is private
  end

  public def method4
    # This method is public
  end
end
```

What's happening here is that the def method2 statement is returning the value :method2, which is immediately being passed as an argument to protected, resulting in protected :method2,